



Webserv

これで、URLがなぜHTTPで始まるのかがようやく理解できるでしょう

概要：

このプロジェクトでは、独自のHTTPサーバーを作成します。

実際のブラウザで動作を確認することができます。

HTTPは、インターネット上で最も広く使われているプロトコルのひとつです。

ウェブサイトの開発に携わらない場合でも、その仕組みを理解することは役立ちます。

バージョン: 24.0

目次

I	はじめに	2
II	一般的なルール	3
III	AIの指示	4
IV	必須項目	6
IV.1	要件	8
IV.2	MacOS専用.....	10
IV.3	設定ファイル	10
V	Readme ファイルの要件	12
VI	ボーナスパート	13
VII	提出と相互評価	14

第1章 はじめに

ハイパーテキスト転送プロトコル（HTTP）は、分散型で協調的なハイパーメディア情報システムのためのアプリケーションプロトコルである。

HTTPは、ワールド・ワイド・ウェブにおけるデータ通信の基盤であり、ハイパーテキスト文書には、ユーザーが簡単にアクセスできる他のリソースへのハイパーリンクが含まれています。例えば、ウェブブラウザ上でマウスボタンをクリックしたり、画面をタップしたりすることでアクセスできます。

HTTPは、ハイパーテキスト機能とワールド・ワイド・ウェブの成長を支えるために開発されました。

Webサーバーの主な機能は、Webページを保存、処理し、クライアントに配信することです。クライアントとサーバー間の通信は、ハイパーテキスト転送プロトコル（HTTP）を通じて行われます。

配信されるページは、ほとんどの場合HTMLドキュメントであり、テキストコンテンツに加えて画像、スタイルシート、スクリプトが含まれていることがあります。

トラフィックの多いウェブサイトでは、複数のWebサーバーが使用され、トラフィックが複数の物理マシンに分割されることがあります。

ユーザーエージェント（通常はウェブブラウザやウェブクローラー）は、HTTPを使用して特定のリソースをリクエストすることで通信を開始し、サーバーはそのリソースの内容、または提供できない場合はエラーメッセージを返します。リソースとは、通常、サーバーのストレージ上に存在する実際のファイル、あるいはプログラムの実行結果を指します。しかし、必ずしもそうとは限らず、実際には他にも多くのものが該当します。

HTTPの主な機能はコンテンツの提供ですが、クライアントがデータを送信することも可能です。この機能は、ファイルのアップロードを含むWebフォームの送信に使用されます。

第2章 一般的なルー

ル

- どのような状況下でも（メモリ不足の場合であっても）、プログラムがクラッシュしたり、予期せず終了したりしてはなりません。
このような事態が発生した場合、プロジェクトは機能していないものとみなされ、成績は0点となります。
- ソースファイルをコンパイルする Makefile を提出しなければなりません。不必要な再リンクを行ってはなりません。
- Makefile には、少なくとも以下のルールが含まれている必要があります。
\$(NAME)、all、clean、fclean、および re。
- コードは c++ および -Wall -Wextra -Werror フラグを使用してコンパイルしてください
- コードはC++98標準に準拠している必要があり、-std=c++98フラグを追加してもコンパイルが成功する必要があります。
- 可能な限り多くのC++の機能を活用してください（例：<string.h>ではなく<cstring>を使用するなど）。C関数の使用は許可されていますが、可能な限りC++版を優先してください。
- 外部ライブラリおよびBoostライブラリの使用は禁止されています。

第 III 章

AI に関する指示

● コンテキスト

学習の過程において、AIはさまざまなタスクを支援してくれます。時間をかけて、AIツールの多様な機能や、それらがどのように自分の仕事をサポートしてくれるかを探ってみてください。ただし、常に慎重に利用し、生成された結果を批判的に評価するように心がけてください。コード、ドキュメント、アイデア、あるいは技術的な説明のいずれであっても、自分の質問が適切に構成されていたか、あるいは生成された内容が正確であるかを完全に確信することはできません。仲間は、間違いや見落としを防ぐための貴重なリソースとなります。

● 主なメッセージ

- ➡ AIを活用して、反復的または単調な作業を減らしましょう。
- ➡ 将来のキャリアに役立つ、コーディングおよび非コーディングの両方のプロンプティングスキルを身につけましょう。
- ➡ AIシステムの仕組みを理解し、よくあるリスク、バイアス、倫理的な問題をよりの確に予測・回避する。
- ➡ 仲間と協力し、技術スキルとリーダーシップスキルの両方を磨き続けましょう。
- ➡ 自分が完全に理解し、責任を負えるAI生成コンテンツのみを使用しましょう。

● 学習者のルール：

- AIツールをじっくりと調べ、その仕組みを理解するようにしましょう。そうすることで、倫理的に活用し、潜在的なバイアスを減らすことができます。
- プロンプトを入力する前に、課題についてよく考えるべきです。そうすることで、正確な語彙を用いて、より明確で、詳細かつ適切なプロンプトを作成できるようになります。
- AIによって生成されたものはすべて、体系的に確認、見直し、疑問を持ち、テストする習慣を身につけるべきです。
- 常にピアレビューを求めるべきです。自分自身の検証だけに頼ってはいけません。4

● フェーズの成果：

- 汎用的なプロンプティングスキルと、特定の分野に特化したプロンプティングスキルの両方を身につけましょう。
- AIツールを効果的に活用して生産性を向上させましょう。
- コンピュータショナル・シンキング、問題解決能力、適応力、および協働能力を引き続き強化する。

● コメントと例：

- 試験や評価など、真の理解を示さなければならない場面に頻繁に直面することになるでしょう。準備を整え、技術的スキルと対人スキルの両方を磨き続けましょう。
- 自分の考えを説明したり、仲間と議論したりすることで、理解の不足が明らかになることがよくあります。仲間との学びを優先しましょう。
- AIツールは、多くの場合、あなたの具体的な状況を理解しておらず、一般的な回答を提示しがちです。同じ環境にいる同僚なら、より適切で正確な洞察を提供してくれるでしょう。
- AIが最も可能性の高い答えを生成する傾向にあるのに対し、同僚は別の視点や貴重なニュアンスを提供してくれます。品質チェックポイントとして、彼らを頼りにしましょう。

✓ ベストプラクティス：

AIに「ソート関数をテストするにはどうすればいいですか？」と尋ねます。すると、いくつかのアイデアが得られます。それらを試してみて、結果を同僚と確認します。そして、二人で協力してアプローチを洗練させていきます。

✗ 悪い例：

AIに関数全体を作成させ、それを自分のプロジェクトにコピペする。ピア評価の際、その関数が何をするのか、なぜそうするのかを説明できない。信頼を失い、プロジェクトは失敗に終わる。

✓ 良い実践例：

AIを活用してパーサーの設計を進めます。その後、同僚と一緒にロジックを確認していきます。そこで2つのバグを見つけ、二人で書き直します。その結果、より良く、よりすっきりとしたコードになり、全員が完全に理解できるようになります。

✗ 悪い例：

プロジェクトの重要な部分のコードをCopilotに生成させた。コンパイルは通ったが、パイプの処理方法を説明できなかった。評価の際、正当な説明ができず、プロジェクトは不合格となった。

第IV章 必須課題

プログラム名	webserv
提出ファイル	Makefile、*.h、*.hpp、*.cpp、*.tpp、*.ipp、 設定ファイル
Makefile	NAME, all, clean, fclean, re
引数	[設定ファイル]
外部関数	すべての機能はC++98で実装されなければなりません。 execve、pipe、strerror、gai_strerror、errno、dup、dup2、 fork、socketpair、htons、htonl、ntohs、ntohl、select、poll、 epoll (epoll_create、epoll_ctl、epoll_wait)、kqueue (kqueue、 kevent)、socket、accept、listen、send、recv、chdir、bind、 connect、getaddrinfo、freeaddrinfo、setsockopt、getsockname 、getprotobyname、fcntl、close、read、write、waitpid、kill 、signal、access、stat、open、opendir、readdir、closedir。
Libft 認証済み	該当なし
説明	C++ 98 による HTTP サーバー

C++ 98 を使用して HTTP サーバーを作成してください。

実行ファイルは次のように実行される必要があります:

./webserv [設定ファイル]



課題や評価シートには poll() が記載されていますが、select()、kqueue()、epoll() などの同
等の関数であればどれでも使用可能です。



このプロジェクトを始める前に、HTTP プロトコルを定義する RFC を読み、telnet や NGINX を使ってテストを行ってください。

RFC のすべてを実装する必要はありませんが、RFC を読むことは、必要な機能の開発に役立ちます。

HTTP 1.0を参考として推奨しますが、必須ではありません。

IV.1 要件

- プログラムは、コマンドライン引数として指定されるか、デフォルトのパスにある設定ファイルを使用する必要があります。
- 他のWebサーバーをexecveすることはできません。
- サーバーは常にノンブロッキング状態を維持し、必要に応じてクライアントの切断を適切に処理しなければなりません。
- 非ブロッキングでなければならず、クライアントとサーバー間のすべての I/O 操作 (listen を含む) に対して 1 つの poll() (または同等の関数) のみを使用しなければなりません。
- poll() (または同等の関数) は、読み取りと書き込みの両方を同時に監視しなければなりません。
- poll() (または同等の関数) を経由せずに、読み取りや書き込み操作を行ってはなりません。
- 読み取りまたは書き込み操作を実行した後、errno の値をチェックしてサーバーの動作を調整することは固く禁じられています。
- 通常のディスクファイルに対しては、poll() (または同等の関数) を使用する必要はありません。これらに対する read() および write() は、準備完了通知を必要としません。



データを待機できる I/O (ソケット、パイプ/FIFO など) は、ノンブロッキングでなければならず、単一の poll() (または同等の関数) によって駆動される必要があります。準備が整っていない状態でこれらの記述子に対して read/recv または write/send を呼び出すと、評価は 0 となります。通常のディスクファイルは例外です。

- poll() または同等の関数を使用する場合、関連するすべてのマクロやヘルパー関数 (例: select() 用の FD_SET) を使用できます。
- サーバーへのリクエストが、無期限にハングアップすることは決してあってはなりません。
- サーバーは、選択した標準的なWebブラウザと互換性がある必要があります。
- ヘッダーの比較やレスポンスの処理にはNGINXを使用できます (HTTPバージョンの違いに注意してください)。
- HTTP レスポンスのステータスコードは正確でなければなりません。
- デフォルトのエラーページが提供されていない場合、サーバーにはデフォルトのエラーページが用意されている必要があります。
- CGI (PHP、Python など) 以外の目的で fork を使用することはできません。
- 完全に静的な Web サイトを配信できる必要があります。
- クライアントはファイルをアップロードできる必要があります。
- 少なくともGET、POST、DELETEメソッドが必要です。

- サーバーが常に利用可能な状態であることを確認するために、ストレステストを行ってください。
- サーバーは、異なるコンテンツを配信するために複数のポートをリッスンする必要があります（[設定ファイル](#)を参照）。



私たちは意図的に、HTTP RFCの一部のみを提供することを選択しました。この文脈において、パーティクルホスト機能は対象外とみなされます。ただし、ご希望であれば実装しても構いません。

IV.2 MacOSのみ



macOSではwrite()の処理が他のUnix系OSとは異なるため、fcntl()の使用が許可されています。他のUnix系OSと同様の動作を実現するには、ファイルディスクリプタをノンブロッキングモードで使用する必要があります。



ただし、fcntl() は以下のフラグでのみ使用できます：
F_SETFL、O_NONBLOCK、および FD_CLOEXEC。
その他のフラグは使用できません。

IV.3 設定ファイル



NGINX設定ファイルの「server」セクションを参考にしてください。

設定ファイルでは、以下のことが可能です：

- サーバーがリスニングするすべてのインターフェースとポートの組み合わせを定義する（プログラムが提供する複数のWebサイトを定義する）。
- デフォルトのエラーページを設定する。
- クライアントのリクエスト本文の最大許容サイズを設定する。
- Webサイトに対して、URL/ルートごとのルールや設定を以下の中から指定できます（ここでは正規表現は不要です）：
 - そのルートで許可されるHTTPメソッドのリスト。
 - HTTPリダイレクト。
 - リクエストされたファイルが配置されるべきディレクトリ（例：URL /kapouet が /tmp/www をルートとする場合、URL /kapouet/pouic/toto/pouet は /tmp/www/pouic/toto/pouet を検索します）。
 - ディレクトリ一覧の表示を有効または無効にします。
 - 要求されたリソースがディレクトリである場合に提供するデフォルトのファイル。
 - クライアントからサーバーへのファイルのアップロードが許可されており、保存場所が指定されています。

- ファイル拡張子（例：.php）に基づいてCGIを実行します。CGIに関する具体的な注意事項は以下の通りです：

- * CGIとは何か、ご存じですか？
- * WebサーバーとCGIの通信に関わる環境変数をよく確認してください。クライアントから提供されたリクエスト全体と引数は、CGIから利用可能でなければなりません。
- * チャンク化されたリクエストの場合、サーバー側でチャンクを解除する必要があることに注意してください。CGIは、ボディの終了としてEOFを期待します。
- * これはCGIの出力についても同様です。CGIからcontent_lengthが返されない場合、EOFが返されたデータの終了を示すことになります。
- * 相対パスによるファイルアクセスを行うため、CGIは適切なディレクトリで実行される必要があります。
- * サーバーは、少なくとも1つのCGI（php-CGI、Pythonなど）をサポートしている必要があります。

評価期間中にすべての機能が正常に動作することをテストおよび実証するために、設定ファイルとデフォルトファイルを提供する必要があります。

ファイルには、その他のルールや設定情報を記述することもできます（たとえば、バーチャルホストを実装する場合のWebサイトのサーバー名など）。



特定の動作について疑問がある場合は、ご自身のプログラムの動作とNGINXの動作を比較してみてください。

簡単なテスターを用意しています。ブラウザやテストで問題なく動作している場合は、必ずしも使用する必要はありませんが、バグの発見や修正に役立ちます。



耐障害性が重要です。サーバーは常に稼働し続けなければなりません。



1つのプログラムだけでテストを行わないでください。PythonやGolangなど、より適切な言語でテストを作成してください。好みに応じてCやC++を使用しても構いません。

第5章

Readmeの要件

Gitリポジトリのルートディレクトリには、README.md ファイルを必ず配置してください。このファイルの目的は、プロジェクトに不慣れな人（同僚、スタッフ、採用担当者など）が、プロジェクトの内容、実行方法、および関連情報の入手先を素早く理解できるようにすることです。

README.md には、少なくとも以下の内容を含める必要があります：

- 最初の行はイタリック体で、次のように記述してください：このプロジェクトは、`<login1>[、<login2>[、<login3>[...]]]` による 42 カリキュラムの一環として作成されました。
- プロジェクトの目的や概要を明確に説明した「Description」セクション。
- コンパイル、インストール、および／または実行に関する関連情報を記載した「Instructions」セクション。
- トピックに関連する主要な参考文献（ドキュメント、記事、チュートリアルなど）を列挙し、AIがどのように使用されたか（どのタスクやプロジェクトのどの部分で使用されたかを明記）を記述した「Resources」セクション。

➡ プロジェクトによっては、追加のセクション（使用例、機能一覧、技術的な選択など）が必要になる場合があります。

必要な追加項目は、以下に明示的に記載します。



READMEは英語で記述してください。

第6章 ボーナスパー

ト

以下に、実装できる追加機能の例を挙げます：

- クッキーとセッション管理のサポート（簡単な例を提示してください）。
- 複数の CGI タイプに対応する。



ボーナス課題は、必須課題に問題なく完全に完了した場合にのみ評価されます。必須要件をすべて満たしていない場合、ボーナス課題は評価されません。

第VII章

提出とピア評価

課題は、通常通りGitリポジトリに提出してください。発表会では、リポジトリの内容のみが評価の対象となります。ファイル名が正しいか、必ず再確認してください。

評価の際、プロジェクトの軽微な修正が求められる場合があります。これには、動作のわずかな変更、数行のコードの追加や書き換え、あるいは簡単に追加できる機能などが含まれます。

この手順はすべてのプロジェクトに当てはまるわけではありませんが、評価ガイドラインに記載されている場合は、その準備をしておく必要があります。

このステップは、プロジェクトの特定の部分について、あなたが実際に理解しているかどうかを確認するためのものです。変更作業は、任意の開発環境（例：普段使用している環境）で行って構いません。また、評価の一環として特定の時間制限が設けられていない限り、数分以内に完了できるものであれば問題ありません。

例えば、関数やスクリプトの軽微な更新、表示の修正、新しい情報を格納するためのデータ構造の調整などが求められる場合があります。

詳細（範囲、対象など）は評価ガイドラインに明記されており、同じプロジェクトであっても評価ごとに異なる場合があります。